Guide til studerende: GitHub, Black Box-test, CI/CD og aflevering i itslearning

Denne guide beskriver hele arbejdsforløbet i caseopgaven: hvordan projektet lægges i GitHub, hvordan black box-test gennemføres, hvordan der laves en CI/CD-pipeline, og hvordan afleveringen og peer feedback håndteres i itslearning. GitHub-repositories kan bruges til at versionsstyre eksisterende projekter, og workflows i GitHub Actions placeres i `.github/workflows/`.[cite:80] GitHub Pages kan bruges til deployment af statiske sider som browserklienten i denne case.[cite:79]

1. Klargør projektet

1. Download og udpak den udleverede zip-fil med projektet.
2. Åbn projektmappen i Visual Studio Code eller i en terminal.
3. Læs filen `materialer/student-opgave.md`, så opgaven og afleveringskravene er klare fra starten.
4. Kontrollér at projektet indeholder C++-kode, testmappe, browserklient og CMake-filer.

2. Opret et GitHub-repository

1. Log ind på GitHub.
2. Opret et nyt repository, for eksempel `quality-api-case`.[cite:80]
3. Vælg et tomt repository uden automatisk README, hvis projektet allerede ligger lokalt.
4. Kopiér repository-URL'en fra GitHub.

3. Initialisér Git lokalt

Åbn en terminal i projektmappen og kør:

```
git init
```

Dette opretter et lokalt Git-repository, som derefter kan forbindes til GitHub med en remote og pushes første gang.[cite:74] [cite:77]

4. Opret `.gitignore`

Før første commit skal der laves en `.gitignore` i projektets rodmappe. GitHub anbefaler at lægge `.gitignore` i repository-roden, så ignore-regler deles med alle, der kloner projektet.[cite:89]

Opret filen `.gitignore` med mindst dette indhold:

```
build/
```

En mere robust version kan være:

```
build/
out/
CMakeCache.txt
CMakeFiles/
CTestTestfile.cmake
cmake_install.cmake
Testing/
compile_commands.json
```

Det er almindelig praksis at ignorere CMake-buildmapper og andre genererede filer, fordi de kan genskabes lokalt og derfor normalt ikke skal versionsstyres.[cite:84] [cite:87] [cite:90]

5. Lav første commit og push til GitHub

Når `.gitignore` er oprettet, kan projektet lægges i GitHub:

```
git add .
git commit -m "Initial commit - imported case project"
git branch -M main
git remote add origin https://github.com/DIT-BRUGERNAVN/quality-api-case.git
git push -u origin main
```

Denne fremgangsmåde svarer til den normale proces for at lægge et eksisterende lokalt projekt op i et nyt GitHub-repository.[cite:74] [cite:77]

6. Byg projektet lokalt

Projektet er et CMake-baseret C++-projekt, og moderne CMake bruges ofte sammen med afhængighedshåndtering som `FetchContent`.[cite:64] I dette projekt bruges GoogleTest sammen med CMake til de automatiske tests.[cite:54] [cite:62]

Kør disse kommandoer i terminalen:

```
cmake -S . -B build
cmake --build build
```

Hvis projektet bygger korrekt, bliver buildfilerne placeret i `build/`, som ikke skal pushes, fordi mappen ligger i `.gitignore`.

7. Start backend lokalt

Når build er gennemført, startes backend-serveren med:

```
./build/quality_api
```

Serveren skal køre, mens browserklienten bruges til black box-test.

8. Gennemfør black box-test

1. Åbn `web/index.html` i en browser.
2. Test systemet uden først at læse kildekoden.
3. Brug ækvivalenspartitionering til at opdele input i gyldige og ugyldige klasser.
4. Brug grænseværdi-analyse omkring værdier som 60, 100, 500, 1000, 5 og 6.[cite:65]
5. Test også kombinationer af parametre som `production mode`, `maintenance mode`, `coupon`, `loyal customer` og sensorværdier.

Black box-test og boundary value analysis bruges netop til at finde fejl omkring gyldige og ugyldige inputgrænser, men skjulte kombinationsfejl kan kræve mere systematisk test. [cite:65]

9. Skriv defect reports

Brug den udleverede defect-template til at registrere fejl. En god fejlrapport bør mindst beskrive defect ID, steps to reproduce, expected result og actual result.[cite:29]

For hver fundet fejl skal der som minimum stå:

- Defect ID
- Titel
- Modul eller endpoint
- Preconditions
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Test type

10. Kør unit tests lokalt

Når den manuelle test er afsluttet, køres de automatiske tests med CTest:

```
ctest --test-dir build --output-on-failure
```

GoogleTest kan integreres i CMake og køres gennem `ctest`, hvilket gør det velegnet til automatisering i CI/CD.[cite:54] [cite:62]

11. Tilføj eller ret tests

1. Åbn testfilen i projektet.
2. Tilføj mindst 3 nye testcases for fejl, der enten er fundet manuelt, eller som mistænkes på baggrund af kravene.
3. Byg igen og kør testene igen:

```
cmake --build build
ctest --test-dir build --output-on-failure
```

Målet er at omsætte manuelle fund til reproducerbare automatiske tests.

12. Opret workflow til GitHub Actions

GitHub Actions-workflows placeres i `.github/workflows/`.[cite:80] Opret derfor mappen og workflow-filen:

```
mkdir -p .github/workflows
```

Opret filen `.github/workflows/ci-cd.yml` med dette indhold:

```
name: Cpp CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Install build tools
        run: sudo apt-get update && sudo apt-get install -y cmake g++ ninja-build

      - name: Configure project
        run: cmake -S . -B build
```

```
      - name: Build project
        run: cmake --build build

      - name: Run tests
        run: ctest --test-dir build --output-on-failure

  deploy-pages:
    if: github.ref == 'refs/heads/main'
    needs: build-and-test
    runs-on: ubuntu-latest
    environment:
      name: github-pages
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload web artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./web

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

GitHub Pages kan deployes med de officielle Pages-actions, og statiske sider som HTML-klienter er en naturlig kandidat til denne type deployment.[cite:79]

13. Commit og push pipeline

Når workflow-filen er oprettet, skal den lægges i GitHub:

```
git add .github/workflows/ci-cd.yml .gitignore
git commit -m "Add CI/CD pipeline and gitignore"
git push
```

Når der pushes til `main`, starter workflowet automatisk i GitHub Actions.[cite:80]

14. Aktivér GitHub Pages

1. Gå til repository på GitHub.
2. Vælg **Settings**.
3. Vælg **Pages**.
4. Sæt source til **GitHub Actions**.[cite:79]

GitHub Pages bruges her kun til deployment af den statiske browserklient. Selve C++-backenden kører ikke på GitHub Pages, fordi Pages er beregnet til statiske sites.[cite:79]

15. Kontroller build, test og deployment

Når workflowet er kørt:

1. Gå til fanen **Actions** på GitHub.
2. Kontrollér at `build-and-test` er grøn.
3. Kontrollér at `deploy-pages` er grøn på `main`.
4. Åbn GitHub Pages-linket og verificér at browserklienten vises.

Hvis testene fejler, skal fejlmeddelelserne bruges til at rette kode eller tests, hvorefter ændringerne commit'es og pushes igen.

16. Aflevering i itslearning

Afleveringen skal afleveres i **itslearning** inden den angivne deadline. Assignment-værktøjet i itslearning understøtter både aflevering, peer response og feedbackfunktioner.[cite:100]

Afleveringen skal indeholde:

- Link til GitHub-repository.
- Udfyldte defect reports.
- Opdaterede eller nye GoogleTest-tests.
- Kort forklaring af pipeline med build, test og deployment.
- Kort refleksion over forskellen mellem manuel black box-test og automatiseret test i CI/CD.

17. Peer feedback i itslearning

itslearning har en indbygget funktion til peer assessment i Assignment-værktøjet.[cite:98] Når underviseren aktiverer **Require peer assessment** og sætter en deadline, bliver afleveringerne automatisk fordelt mellem studerende, efter at deadline er passeret.[cite:98]

Det betyder i praksis:

1. Hver studerende afleverer sin egen opgave i itslearning inden deadline.
2. Efter deadline fordeler itslearning automatisk det valgte antal besvarelser til peer assessment blandt de studerende, der har afleveret.[cite:98]
3. Studerende, der ikke har afleveret, indgår ikke i peer assessment-processen.[cite:98]
4. Den enkelte studerende vil derefter se opgaven igen i sin opgaveliste med handlingen **assess peers**.[cite:98]
5. Feedback gives direkte i itslearning, hvor man kan skrive samlet feedback i teksteditoren, bruge rubric hvis underviseren har tilføjet en, og i mange tilfælde annotere dokumenter i systemets dokumentviser.[cite:98]
6. itslearning har også en funktion som **Save and assess next**, så man nemt kan gå videre til næste tildelte besvarelse.[cite:98]

18. Krav til feedbacken

Feedbacken til medstuderende skal være saglig, konkret og konstruktiv. Den skal tage udgangspunkt i opgavens faglige mål og bør både pege på styrker i løsningen og på mulige forbedringer.

Feedbacken bør som minimum forholde sig til:

- Om GitHub-repository og struktur er sat fornuftigt op.
- Om `.gitignore` er brugt korrekt.
- Om black box-testen virker systematisk og velbegrundet.
- Om defect reports er klare og reproducerbare.
- Om CI/CD-pipelinen giver mening med build, test og deployment.
- Om forklaringerne er tydelige og fagligt korrekte.

19. Vigtig note om peer assessment

Peer assessment i itslearning kræver en deadline, fordi systemet først fordeler besvarelserne efter deadline er udløbet.[cite:98] Fordelingen sker typisk nogle få minutter efter deadline, blandt andet fordi itslearning arbejder med en kort soft grace-periode omkring afleveringstidspunktet.[cite:98]

Hvis underviseren vælger anonym peer assessment, kan identiteten på den afleverende studerende skjules for medstuderende under feedbackprocessen.[cite:98]

20. Faglig pointe

Denne case viser overgangen fra manuel test til automatiseret kvalitetssikring. Manuel black box-test er god til at opdage brugerrettede fejl og tydelige afvigelser, mens automatiserede tests og CI/CD er stærke til at gøre test reproducerbar og fange regressioner tidligt i udviklingsforløbet.[cite:54] [cite:65]